Lógica de Programação — Aula 01

O Pensamento Computacional e a Magia do Scratch

João Quentino

26 de Fevereiro de 2026

Jovem Tech — Turma 04A - Manhã

Olá, Mundo!

Instrutor: João Quentino
Data Scientist & Professor

Contato:
joao.quentino@dcomp.ufs.br

Portfolio:
joaoq.vercel.app

Linkedin:
https://www.linkedin.com/in/felipequentino/

Objetivo de hoje:
Entender como as máquinas "pensam"e criar nosso primeiro algoritmo.

Agenda de Hoje

O que é Computação?

Como as Máquinas se Comunicam?

Algoritmos: A Receita do Sucesso

O Modelo Fundamental: IPO

Introdução ao Scratch

Preparando para a Próxima Aula

O que é Computação?

Computação não é sobre Computadores

> *"A Ciência da Computação não trata de computadores, assim como a Astronomia não trata de telescópios."*
> *— Edsger W. Dijkstra*

É sobre Resolução de Problemas:

- **Input:** A informação que entra.
- **Algoritmo:** O processo (a caixa preta).
- **Output:** A solução.

Exemplos do Dia a Dia

Pensamento Computacional está em toda parte!

GPS/Waze:

- Input: Origem e destino
- Algoritmo: Cálculo de menor caminho
- Output: Rota otimizada

Netflix/Spotify:

- Input: Histórico de visualização
- Algoritmo: Machine Learning
- Output: Recomendações

Desafio: Você consegue pensar em mais 3 exemplos?

Como as Máquinas se Comunicam?

O Sistema Binário

Computadores são compostos por bilhões de minúsculos interruptores (transistores).

- **0 (OFF):** Ausência de eletricidade.
- **1 (ON):** Presença de eletricidade.

Binário	Decimal	Representação Visual
000	0	○ ○ ○
001	1	○ ○ ●
010	2	○ ● ○
011	3	○ ● ●
100	4	● ○ ○
101	5	● ○ ●
110	6	● ● ○
111	7	● ● ●

Como Funciona a Conversão?

Sistema Decimal (Base 10):

$$234 = 2 \times 10^2 + 3 \times 10^1 + 4 \times 10^0$$

Sistema Binário (Base 2):

$$101_2 = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 4 + 0 + 1 = 5_{10}$$

Cada posição vale o dobro da anterior!

128	64	32	16	8	4	2	1
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

Exercício Prático: Conversão Binária

Converta para decimal:

1. $1010_2 = ?$
2. $1111_2 = ?$
3. $10000_2 = ?$

Dica: Some apenas as posições que têm o bit 1 ligado!

Exercício Prático: Conversão Binária

Converta para decimal:

1. $1010_2 = ?$
2. $1111_2 = ?$
3. $10000_2 = ?$

Dica: Some apenas as posições que têm o bit 1 ligado!

Respostas:

1. $1010_2 = 8 + 2 = 10_{10}$
2. $1111_2 = 8 + 4 + 2 + 1 = 15_{10}$
3. $10000_2 = 16_{10}$

Representando Informação

- **Números:** Através do sistema de potências de 2.
- **Letras (ASCII/Unicode):**
  - O número 65 representa a letra 'A'.
  - O número 97 representa a letra 'a'.
  - `'H' 'e' 'l' 'l' 'o'` = `72 101 108 108 111`
- **Imagens:** Pixels são trios de números (RGB).
  - Vermelho puro: (255, 0, 0)
  - Verde puro: (0, 255, 0)
  - Azul puro: (0, 0, 255)

Conceito Chave: Tudo no computador é, no fundo, um número binário.

Bits, Bytes e Além

Unidades de Medida:

- **1 Bit:** Menor unidade (0 ou 1)
- **1 Byte:** 8 bits (pode representar 0-255)
- **1 Kilobyte (KB):** 1.024 bytes
- **1 Megabyte (MB):** 1.024 KB $\approx$ 1 milhão de bytes
- **1 Gigabyte (GB):** 1.024 MB $\approx$ 1 bilhão de bytes

Curiosidade: Uma foto do seu celular pode ter 3-5 MB.
Quantos bytes são necessários para armazenar essa foto?

Bits, Bytes e Além

Unidades de Medida:

- **1 Bit:** Menor unidade (0 ou 1)
- **1 Byte:** 8 bits (pode representar 0-255)
- **1 Kilobyte (KB):** 1.024 bytes
- **1 Megabyte (MB):** 1.024 KB $\approx$ 1 milhão de bytes
- **1 Gigabyte (GB):** 1.024 MB $\approx$ 1 bilhão de bytes

Curiosidade: Uma foto do seu celular pode ter 3-5 MB. Quantos bytes são necessários para armazenar essa foto?

Resposta: Aproximadamente 3-5 milhões de bytes!

Algoritmos: A Receita do Sucesso

O que é um Algoritmo?

Definição: Um algoritmo é uma sequência finita e bem definida de passos para resolver um problema ou realizar uma tarefa.

Características de um bom algoritmo:

- **Finito:** Deve terminar em algum momento.
- **Definido:** Cada passo deve ser claro e sem ambiguidade.
- **Entrada:** Pode ter zero ou mais entradas.
- **Saída:** Deve produzir pelo menos uma saída.
- **Efetivo:** Deve ser executável com recursos disponíveis.

O Algoritmo do Sanduíche (Inspirado no CS50)

Se eu pedisse para você me ensinar a fazer um sanduíche de manteiga de amendoim, e eu fosse um robô "burro", o que aconteceria?

1. Pegue o pão. (Como? Com a mão? Com força?)
2. Passe a manteiga. (Com o quê? A faca está fechada na gaveta!)

Lição: Computadores precisam de instruções **precisas** e **sequenciais**.

Algoritmo Melhorado: Fazer um Sanduíche

Versão Detalhada:

1. Localize o pacote de pão na despensa
2. Abra o pacote girando o lacre
3. Remova duas fatias de pão
4. Coloque as fatias lado a lado na bancada
5. Localize o pote de manteiga de amendoim
6. Abra o pote girando a tampa no sentido anti-horário
7. Pegue uma faca da gaveta
8. Insira a faca no pote
9. Retire uma porção de manteiga (aproximadamente 2 colheres de sopa)
10. Espalhe uniformemente sobre uma fatia
11. Coloque a segunda fatia em cima
12. O sanduíche está pronto!

Algoritmos do Cotidiano

Exemplos que você conhece:

Receita de Bolo:

1. Pré-aqueça o forno
2. Misture ingredientes secos
3. Adicione ingredientes líquidos
4. Bata por 5 minutos
5. Asse por 40 minutos

Atravessar a Rua:

1. Pare na calçada
2. Olhe para a esquerda
3. Olhe para a direita
4. Se não vem carro, atravesse
5. Caso contrário, espere

Programação é traduzir esses algoritmos para linguagem de máquina!

Pseudocódigo: A Ponte Entre Ideias e Código

O que é? Uma descrição informal de um algoritmo usando linguagem humana estruturada.

Exemplo: Encontrar o maior número

Pseudocódigo

```
INÍCIO
  Defina maior = primeiro_número
  PARA CADA número na lista:
    SE número > maior ENTÃO
      maior = número
    FIM SE
  FIM PARA
  Exiba maior
FIM
```

Atividade: Crie seu Algoritmo!

Desafio em Dupla (5 minutos):

Escreva o algoritmo detalhado para:

1. **Grupo 1-3:** Como escovar os dentes (assuma que sou um robô!)
2. **Grupo 4-6:** Como fazer uma ligação de celular
3. **Grupo 7-9:** Como preparar um copo de suco

Critérios:

- Seja MUITO específico
- Não assuma conhecimento prévio
- Cada passo deve ser executável

Vamos apresentar e testar se o "robô"consegue executar!

O Modelo Fundamental: IPO

Input → Processo → Output

Todo programa segue este modelo:

Dados Brutos
Informação inicial

Algoritmo
Transformação

Resultado
Informação útil

Exemplos Práticos do Modelo IPO

1. **Calculadora:**
   - Input: $5 + 3$
   - Processo: Soma
   - Output: 8
2. **Google Tradutor:**
   - Input: "Hello"
   - Processo: Tradução (algoritmo de NLP)
   - Output: "Olá"
3. **Filtro do Instagram:**
   - Input: Foto original
   - Processo: Ajustes de cor/saturação
   - Output: Foto filtrada

Aplicando IPO em Problemas Reais

Exercício Mental: Identifique Input, Processo e Output

1. **Sistema de Notas da Escola**
2. **Caixa Eletrônico (ATM)**
3. **Sistema de Alarme**

Aplicando IPO em Problemas Reais

Exercício Mental: Identifique Input, Processo e Output

1. **Sistema de Notas da Escola**
2. **Caixa Eletrônico (ATM)**
3. **Sistema de Alarme**

Possíveis Respostas:

- **Notas:** Input = provas/trabalhos — Processo = média — Output = nota final
- **ATM:** Input = cartão+senha — Processo = validação — Output = dinheiro/recibo
- **Alarme:** Input = sensor de movimento — Processo = verificação — Output = sirene

Introdução ao Scratch

Por que Scratch?

- Criado pelo MIT Media Lab (2007)
- Remove a barreira da **sintaxe** (erros de digitação)
- Foca puramente na **lógica**
- Programação **visual** por blocos
- Mais de 100 milhões de projetos criados!

Filosofia: "Imagine, Programe, Compartilhe"

Grandes programadores começaram com ferramentas simples!

A Interface do Scratch

Principais Áreas:

1. **Palco (Stage)**
   - Onde acontece a ação
   - 480 x 360 pixels
   - Coordenadas (x, y)
2. **Área de Sprites**
   - Personagens e objetos
   - Cada um tem seu script
3. **Blocos de Código**
   - Movimento
   - Aparência
   - Som
   - Eventos
   - Controle
   - Sensores
   - Operadores
   - Variáveis

Vocabulário de Programação no Scratch

Sprites: Personagens ou objetos na tela.

Functions (Funções): Blocos que executam uma ação (ex: *"mova 10 passos"*).

Conditions (Condicionais): Tomada de decisão (ex: *"se tocar na borda, então..."*).

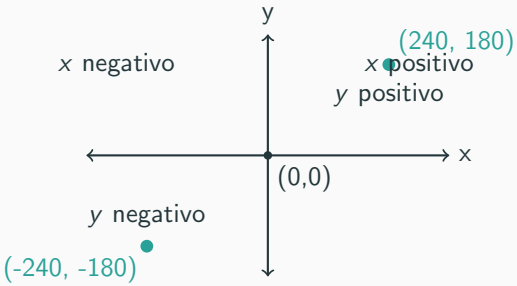
Loops (Laços): Repetição de tarefas (ex: *"sempre"*).

Events (Eventos): O gatilho para a ação (ex: *"quando a bandeira for clicada"*).

Variables (Variáveis): Caixas que guardam informações.

Sistema de Coordenadas

O palco usa o plano cartesiano:



Importante: Centro = (0, 0) — Canto superior direito = (240, 180)

Blocos Fundamentais

Categorias Essenciais:

1. **Eventos (Amarelo):** Quando iniciar
   - "quando bandeira verde clicada"
   - "quando tecla espaço pressionada"
2. **Movimento (Azul):** Controle de posição
   - "mova 10 passos"
   - "vá para x: 0 y: 0"
3. **Controle (Laranja):** Lógica de execução
   - "sempre"
   - "se... então"
   - "repita 10 vezes"

Sequenciamento: A Ordem Importa!

Conceito: Instruções são executadas de cima para baixo.

Exemplo - O que acontece?

Script A:

1. Vá para x: 0 y: 0
2. Mova 100 passos
3. Diga "Cheguei!"

Script B:

1. Diga "Cheguei!"
2. Mova 100 passos
3. Vá para x: 0 y: 0

Resultado diferente! A ordem dos blocos determina o comportamento.

Desafio Prático 1: O Primeiro Script

Atividade Guiada (10 min):

1. Acesse `scratch.mit.edu`
2. Faça login ou escolha "Criar"
3. Faça o gato dizer "Olá, Jovem Tech!" por 2 segundos
4. Faça-o mudar de cor
5. Faça-o dar 10 passos para frente

Dica: Use blocos de:

- Eventos (bandeira verde)
- Aparência (diga, mude cor)
- Movimento (mova passos)

Desafio Prático 2: Movimento Contínuo

Atividade (15 min):

1. Faça o gato andar infinitamente de um lado para o outro
2. Use o bloco "sempre"(loop infinito)
3. Use "se tocar na borda, volte"para inverter direção
4. **BÔNUS:** Faça-o mudar de fantasia ao virar

Conceitos Aplicados:

- Loop infinito (sempre)
- Condicional implícito (tocar na borda)
- Movimento sequencial

Parabéns! Você acabou de criar seu primeiro algoritmo animado!

Desafio Prático 3: Controle por Teclado

Atividade Avançada (15 min):

Objetivo: Criar um sprite que se move com as setas do teclado.

Instruções:

- Use 4 blocos "quando tecla ... pressionada"
- Seta para cima: mude y em 10
- Seta para baixo: mude y em -10
- Seta para direita: mude x em 10
- Seta para esquerda: mude x em -10

Desafio Extra:

- Faça o sprite apontar na direção do movimento
- Adicione um segundo sprite que se move sozinho

Debugging: Encontrando Erros

O que é Debugging?

- Processo de identificar e corrigir erros (bugs)
- Nome vem de um inseto real que quebrou um computador em 1947!

Dicas para Debug no Scratch:

1. Execute o código passo a passo (modo turbo OFF)
2. Use blocos "diga" para verificar valores
3. Teste cada parte isoladamente
4. Verifique se os blocos estão conectados
5. Compare com exemplos que funcionam

"Depurar código é como ser um detetive em um drama policial!- Anônimo

Preparando para a Próxima Aula

O que vem a seguir?

Na Aula 02, vamos mergulhar em:

- **Variáveis:** Como armazenar e manipular dados
- **Loops Complexos:** Repita até, para cada
- **Condicionais Aninhadas:** Se dentro de se
- **Operadores Lógicos:** E, OU, NÃO
- **Projeto Integrador:** Criar um jogo simples

Para Casa (Opcional):

1. Explore projetos na comunidade Scratch
2. Tente remixar um projeto existente
3. Anote 3 ideias de projetos que gostaria de criar

Recapitulação

Hoje aprendemos:

1. Computação = Resolução sistemática de problemas
2. Binário é a linguagem fundamental das máquinas
3. Algoritmos são sequências precisas de instruções
4. IPO (Input-Processo-Output) é o modelo básico
5. Scratch nos permite programar visualmente
6. Sequenciamento e ordem são cruciais

Habilidades Desenvolvidas:

- Pensamento computacional
- Decomposição de problemas
- Criação de algoritmos visuais

Reflexão Final

"A programação não é sobre tecnologia, é sobre transformar ideias em realidade."

Você acabou de dar o primeiro passo nessa jornada!

Continue explorando, testando e criando!

Dúvidas e Próximos Passos

Fique à vontade para perguntar!

Recursos Úteis:

- **Scratch:** `scratch.mit.edu`
- **Tutoriais:** `scratch.mit.edu/ideas`
- **CS50:** `cs50.harvard.edu`
- **Code.org:** `code.org`

Contato:

`joao.quentino@dcomp.ufs.br`

Nos vemos na próxima aula!